

DIGITAL LOGIC LEARNING PROJECT

8-Bit Processor Design

Architecture, instruction set, and processor verification

CONTRIBUTOR

Barış Şan

Formatted Instructions below:

1

Instruction in natural language: *Load 3 into Register 5*

Instruction in machine code: *li , r5, 3*

Instruction as R or I type in binary format: (I type)

<i>ignore</i>	<i>opcode</i>	<i>Rs</i>	<i>rt</i>	<i>rd</i>	<i>immediate</i>
000	0011	000	000	101	0000 0011

Hex code representation: 060503

2

Instruction in natural language: *Load 2 into Register 7*

Instruction in machine code: *li , r7, 2*

Instruction as R or I type in binary format: (I type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>immediate</i>
000	0011	000	000	111	0000 0010

Hex code representation: 060702

3

Instruction in natural language: *Add Register 7 to Register 5 and put the result in Register 6*

Instruction in machine code: *add , r6,r7, r5*

Instruction as R or I type in binary format: (R type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	0000	111	101	110	0000 0000

Hex code representation: 01EE00

4

Instruction in natural language: Rotate the contents of Register 5 twice and keep the result in Register 4

Instruction in machine code: *rotl , r4,r5, r7*

Instruction as R or I type in binary format: (R type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	1100	101	111	100	0000 0000

Hex code representation: 197C00

5

Instruction in natural language: Save contents of Register 6 at address 2 in data memory

Instruction in machine code: *sb , r6,2*

Instruction as R or I type in binary format: (I type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	0010	000	110	000	0000 0010

Hex code representation: 043002

6

Instruction in natural language: Bit-wise AND Register 4 and Register 6 keep the result in Register 2

Instruction in machine code: *and ,r2, r6,r4*

Instruction as R or I type in binary format: (R type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	0100	110	100	010	0000 0000

Hex code representation: 09A200

7

Instruction in natural language: Grab the value stored in location 2 of data memory and store in Register 3

Instruction in machine code: *lbu,r3, 2*

Instruction as R or I type in binary format: (I type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	0001	000	000	011	0000 0010

Hex code representation: 020302

8

Instruction in natural language: *Bit-wise OR the contents of Register 3 and Register 4 and keep the result in Register 6*

Instruction in machine code: *or,r6, r3,r4*

Instruction as R or I type in binary format: (R type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	1000	011	100	110	0000 0010

Hex code representation: *10E602*

9

Instruction in natural language: *Add Register 5 to Register 2 and put the result in Register 1.*

Instruction in machine code: *add,r1, r5,r2*

Instruction as R or I type in binary format: (R type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	0000	101	010	001	0000 0000

Hex code representation: *015100*

10

Instruction in natural language: *Save the contents of Register 1 to address 5 in data memory.*

Instruction in machine code: *sb,r1, 5*

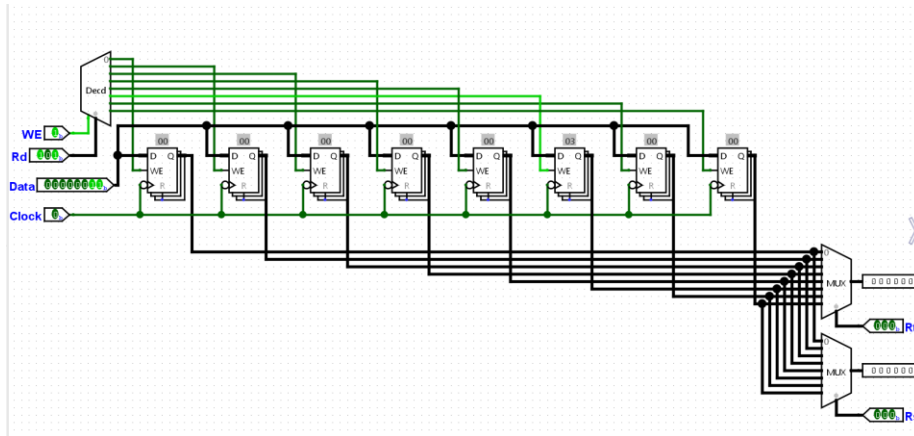
Instruction as R or I type in binary format: (I type)

<i>ignore</i>	<i>opcode</i>	<i>rs</i>	<i>rt</i>	<i>rd</i>	<i>shamt</i>
000	0010	000	001	000	0000 0101

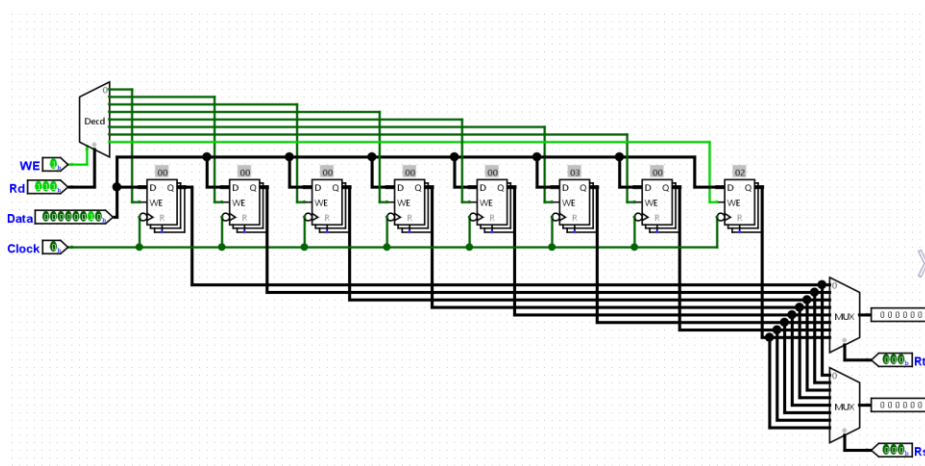
Hex code representation: *040805*

Screenshots of memory elements after per instruction passed:

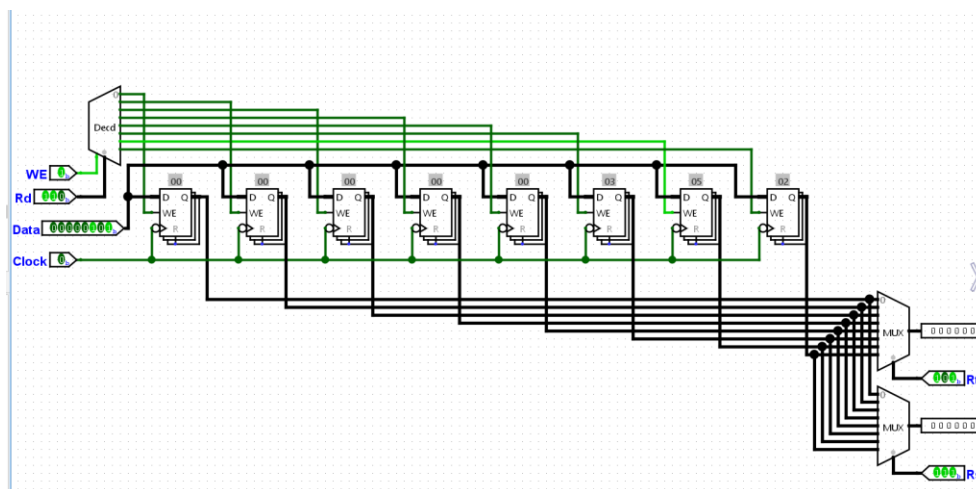
1



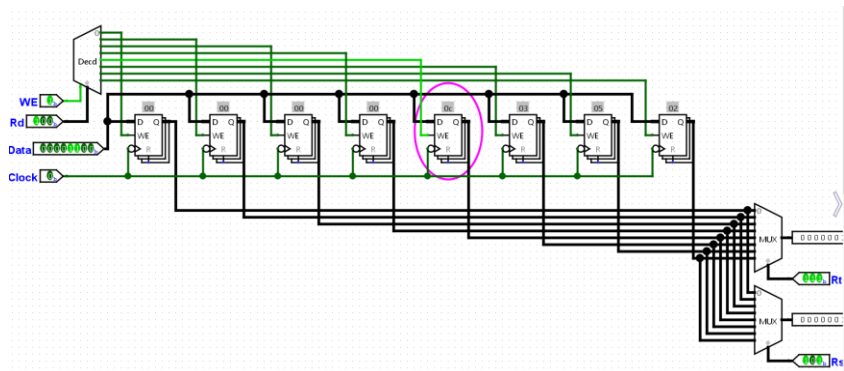
2



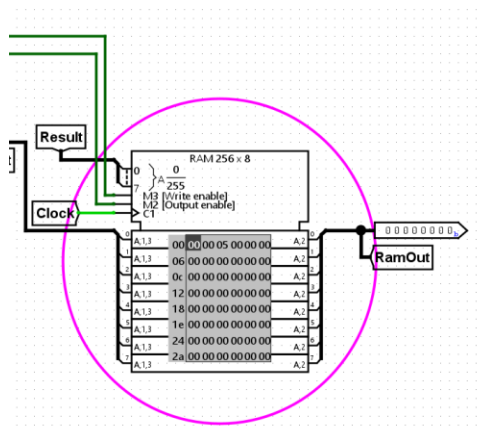
3



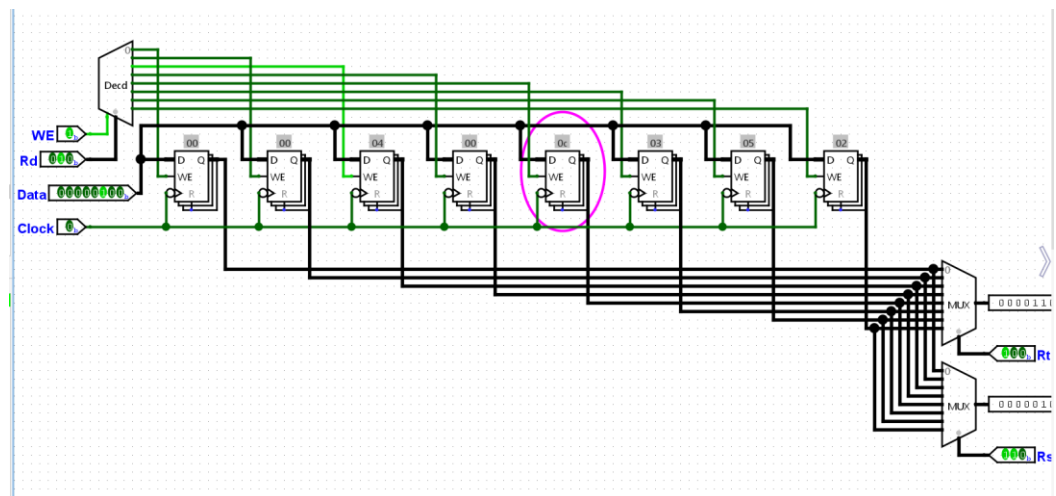
4



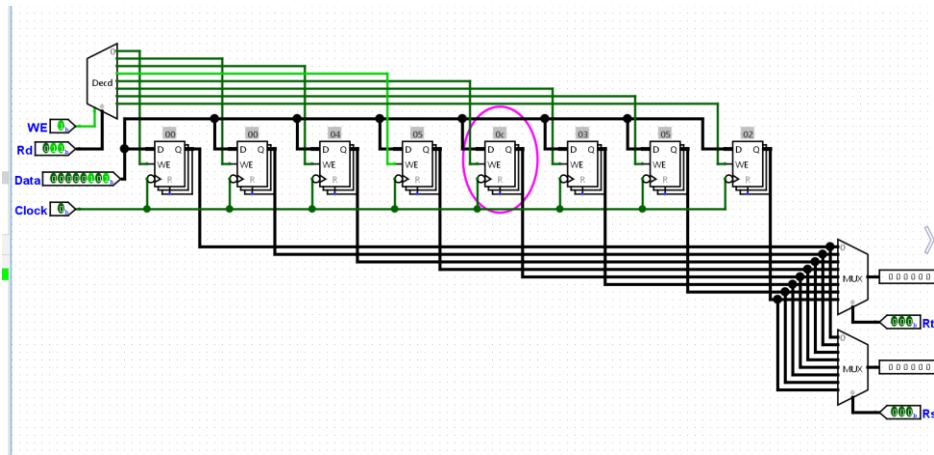
5



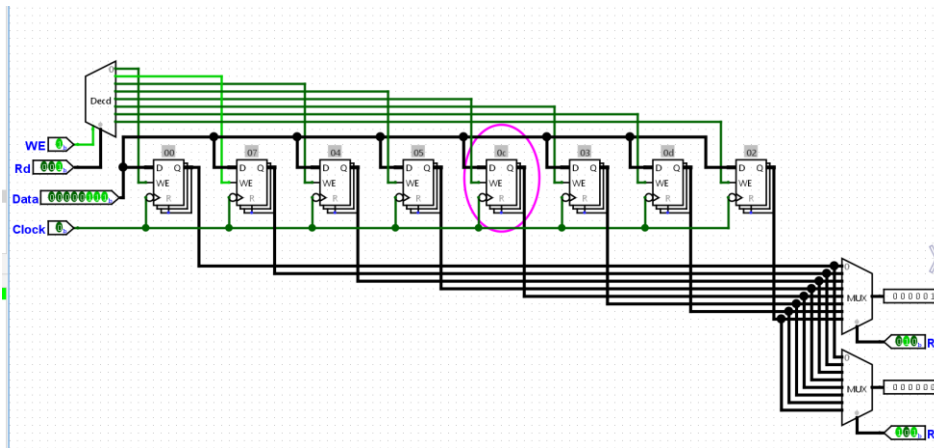
6



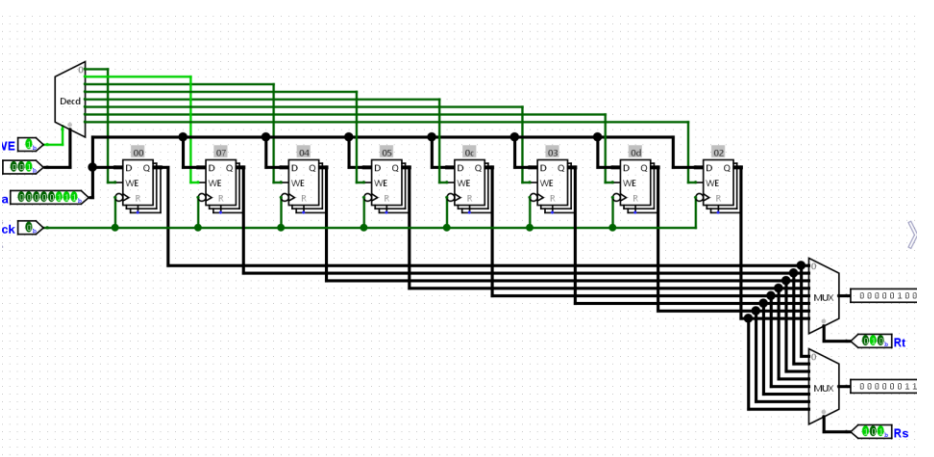
7



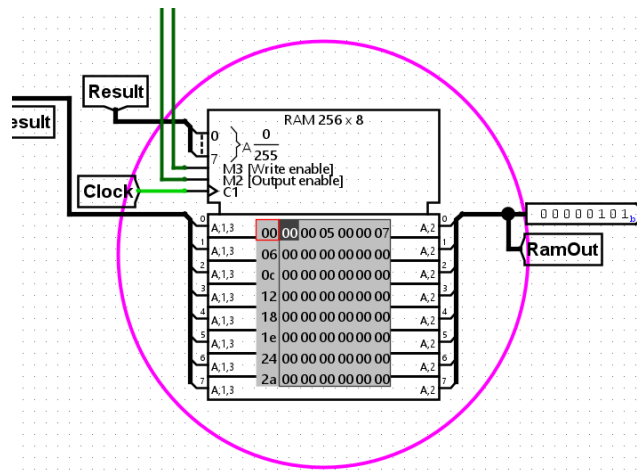
8



9



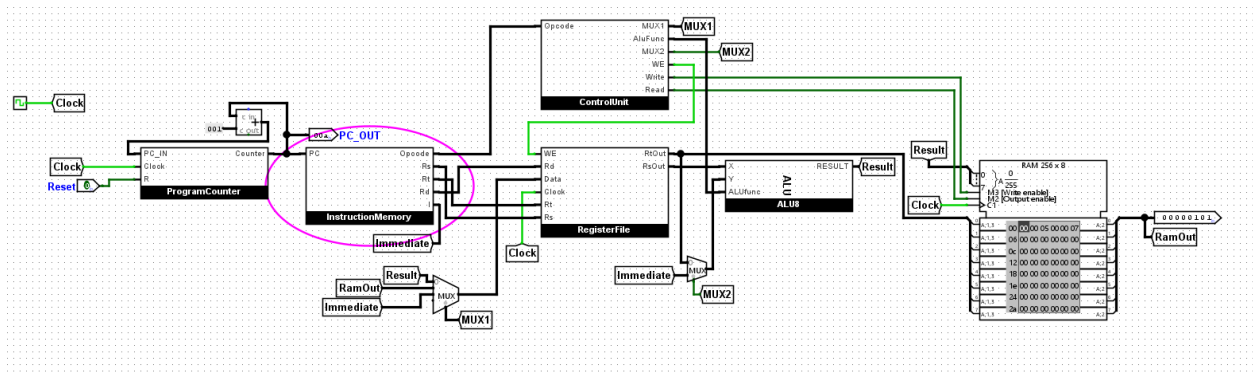
10



Schemas from 8-bit CPU:

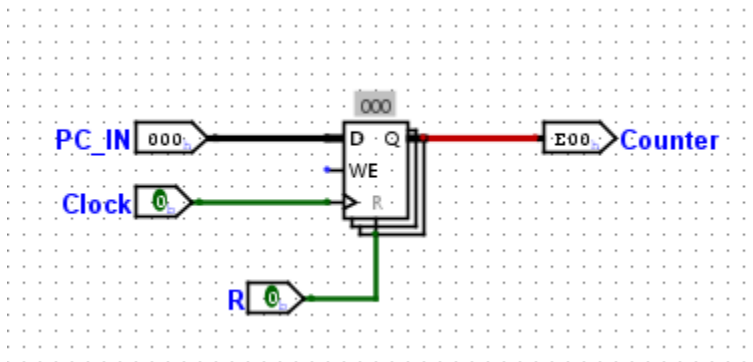
CPU:

This is total form of CPU, I used tunnels to remove complicated looks. And I didn't created any Data memory circuit, I directly place ram on CPU part.



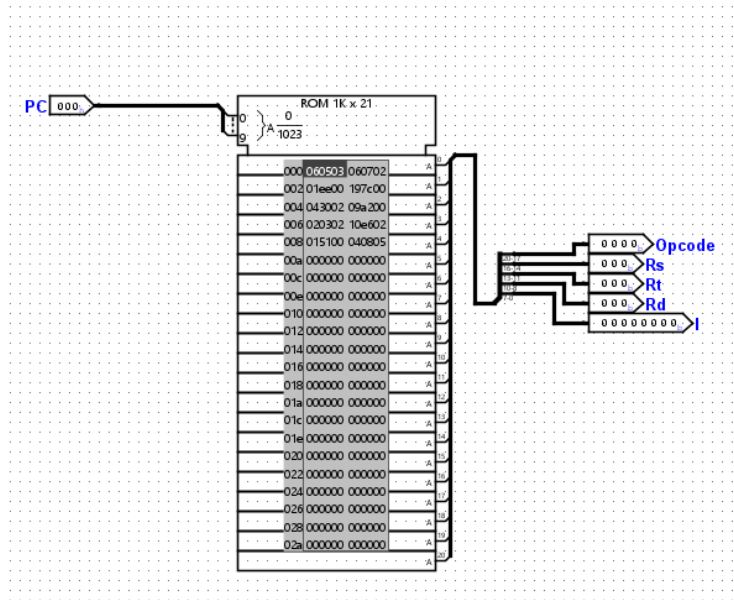
Program Counter

Just a program counter, incrementing part done in CPU. Program counter has 10 digits.



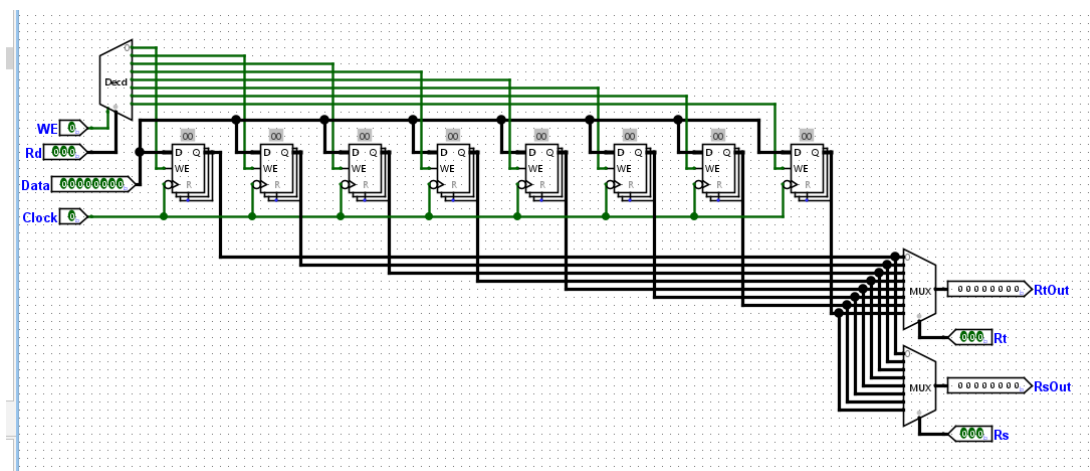
Instruction Memory

Instruction memory is used to store instructions in hexadecimal values. For this report specifically I added 10 instructions manually. While storing 3 bit gone empty and rest of them categorized as: opcode(4-bit), Rs(3-bit), Rt(3-bit), Rd(3-bit), Shamt(8-bit) -> total 21 bit used.



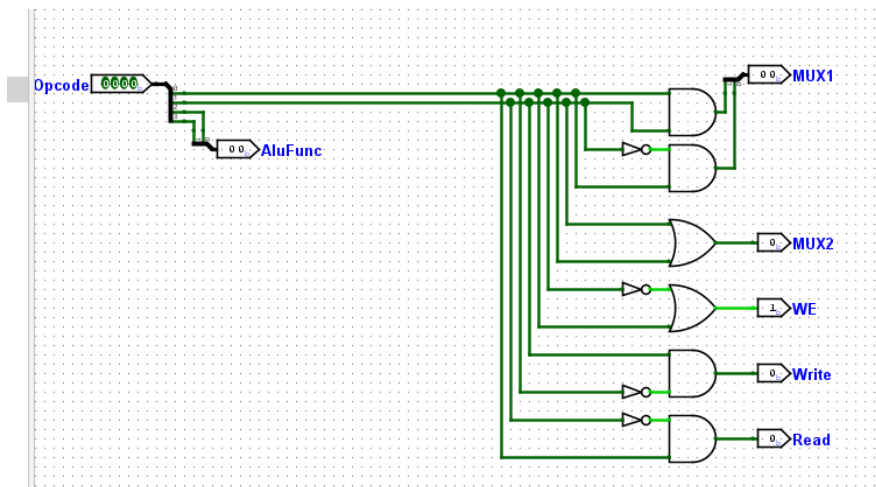
Register File

There are 8 registers for our 8-bit CPU, first one is zero-base register. Rd is register to write, WE is write permission for register and Rt and Rs registers are reads process values from registers for various operations. Data is incoming data to write, which can be output of ALU, RAM(Data memory), value from instruction.



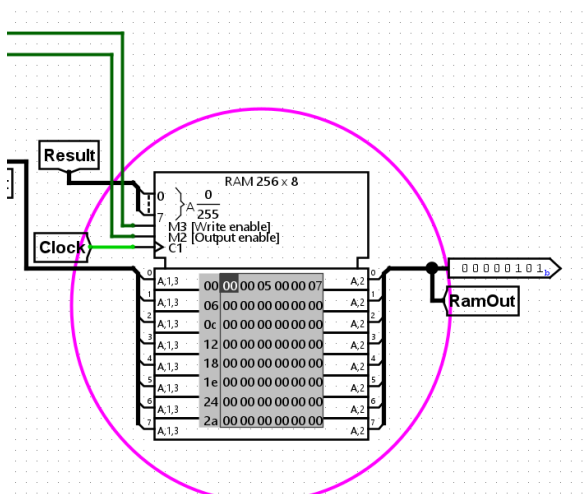
Control Unit

Control unit reads opcode come from instruction and decides ALU operations, managing data's vent to previous circuits or permissions for writing reading memory. It is designed with given equations and tables.



Data memory

Data memory in this project is just implemented RAM. It can not store values without power unlike ROM that we used for instructions memory. Values can be stored and read with permissions come from control unit.



ALU

ALU is does operations, decided from control unit.

